Integrated Final Reference Document – Lab 10 Context, Narrative, Resolved Confusion, File Locations, Evolution of Code Suggestions, and Most Complete Implementations (as of March 2026)

This document merges:

- the storytelling / "why are we even doing this" narrative layer
- the step-by-step environment & confusion resolution layer
- every major code version that appeared during the debugging / guidance conversation
- the final agreed-upon most complete & realistic implementations of the key files

The goal is to give one self-contained artefact that someone opening the conversation months later can read from top to bottom and immediately understand both **the intellectual purpose** and **the concrete technical state** that was reached.

1. The Real Story – What Lab 10 Is Actually Teaching (Narrative Layer)

Lab 10 is **not** a Docker compose exercise dressed up with Ollama and MongoDB.

It simulates building a **minimal runtime behavioural watchdog / lightweight EDR-like component** for untrusted code execution in containers — the kind of system that security product teams at Sysdig, CrowdStrike Falcon for Containers, Tetragon + eBPF, Aqua Security, Lacework, or even internal platform security groups at large cloud/AI companies have been building since ~2022–2025.

Core premise of the world the lab inhabits (2026 perspective):

- Almost all computation is Python (notebooks, agents, research scripts, student assignments, customer plugins, fine-tuning jobs, evaluation harnesses)
- Static analysis (Bandit, Semgrep, pip-audit, dependency-check) catches maybe 30–40% of realistic threats in dynamic Python code
- The really dangerous behaviours only reveal themselves **at runtime**:
    - delayed payloads (sleep 2 days then exfil)
    - environment-triggered logic (if AWS_METADATA available → steal creds)
    - slow & low exfiltration (10 KB every 5 minutes)
    - in-memory code loading / exec / pickle bombs
    - legitimate-looking packages that phone home only after certain conditions
    - resource abuse that looks like normal training until GPU hours are burned

Static tools fail here. Signature-based tools fail here. The only scalable answer left is **runtime telemetry fusion + lightweight behavioural rules + scoring + explainability**.

Lab 10 makes the student implement **exactly that stack** at toy scale:

1. sandbox = untrusted execution zone (exposes /monitor/* and /telemetry/* endpoints)
2. monitor = sidecar watchdog that polls telemetry, computes indicators, scores risk, writes human-readable justification, persists forensic record
3. student = threat modeller / detection engineer → decides which signals matter, how to weigh them, how to explain decisions

The coursework grade is largely a function of:

- richness & relevance of added telemetry features
- thoughtfulness of scoring rules & thresholds
- quality & auditor-readability of generated explanations
- overall coherence of the behavioural model you built

That is why the skeleton is deliberately minimal — the value is in what **you** layer on top.

2. Final Environment & Editing Map (Confusion Fully Resolved)

One truth repeated:

Physical location	**Purpose**	**How to reach it**	**Editing allowed here?**
Host machine (PowerShell / terminal)	docker compose up/down/restart/logs	Just open your normal terminal	No
comp40771_container	**developer workstation** – edit all code, run notebook	docker exec -it comp40771_container bash then cd /opt/data/monitor	**YES – only here**
monitor container	runtime instance of your monitoring logic	Controlled only via host: docker compose restart monitor	No
sandbox container	untrusted code execution target	You never enter or edit it	No

Editing entry points (pick one – browser is strongly preferred):

- Browser (easiest): http://localhost:8081/lab → file tree → /opt/data/monitor/... → double-click any .py file
- Terminal nano (what the screenshot showed):

Bash

cd /opt/data/monitor

nano utils/telemetry.py

3. Key Files – Location, Purpose, Evolution & Most Complete Suggested Versions

3.1 utils/telemetry.py

Original skeleton problem Only had POST helper + very basic start_telemetry logic → could not actually read CPU, network, files, etc.

Most complete version reached in conversation (includes GET, wrappers, timestamps, light enrichment)

Python

```
# /opt/data/monitor/utils/telemetry.py


import requests

import time

from typing import Dict, Any


BASE_URL = "http://sandbox:8000"


def _post(

    endpoint: str,

    data: dict = None,

    timeout: int = 10,

    retries: int = 5

) -> Dict[str, Any]:

    """POST helper – usually already present in skeleton"""
```

```
    url = f"{BASE_URL}{endpoint}"
    for attempt in range(retries):
        try:
            resp = requests.post(url, json=data or {}, timeout=timeout)
            resp.raise_for_status()
            ct = resp.headers.get("content-type", "").lower()
            if "json" in ct:
                return resp.json()
            return {"status": resp.status_code, "text": resp.text.strip()}
        except Exception as exc:
            if attempt == retries - 1:
                raise RuntimeError(f"POST {endpoint} failed after {retries} attempts: {exc}")
            time.sleep(1.1)
    return {"error": "all retries failed"}



def _get(
    endpoint: str,
    timeout: int = 12,
    retries: int = 4
) -> Dict[str, Any]:
    """Critical addition – GET with retries"""
    url = f"{BASE_URL}{endpoint}"
    for attempt in range(retries):
        try:
            resp = requests.get(url, timeout=timeout)
            resp.raise_for_status()
            ct = resp.headers.get("content-type", "").lower()
```

```
            if "json" in ct:
                return resp.json()
            return {"status": resp.status_code, "text": resp.text.strip()}
        except Exception as e:
            if attempt == retries - 1:
                return {"error": str(e), "attempts": attempt + 1}
            time.sleep(1.2)
    return {"error": "all retries failed"}


# Convenience wrappers – called from monitor loop or agents
def get_cpu()         -> dict: return _get("/monitor/cpu")
def get_network()     -> dict: return _get("/monitor/network")
def get_files()       -> dict: return _get("/monitor/files")
def get_threads()     -> dict: return _get("/monitor/threads")
def get_storage()     -> dict: return _get("/monitor/storage")
def get_recent_events() -> dict: return _get("/telemetry/events/recent")


def collect_all_telemetry() -> Dict[str, Any]:
    """Central collection function – most important student contribution"""
    ts = time.time()
    data: Dict[str, Any] = {
        "timestamp_unix": ts,
        "iso_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime(ts)),
        "cpu":         get_cpu(),
        "network":     get_network(),
        "filesystem":  get_files(),
```

```
    "threads":      get_threads(),
    "storage":      get_storage(),
    "recent_events": get_recent_events(),
  }

  # Lightweight feature engineering / indicators (this is where coursework marks live)
  if isinstance(data.get("cpu"), dict):
    pct = data["cpu"].get("system_cpu_percent", 0)
    data["cpu"].update({
      "is_high":     pct > 85,
      "is_critical":  pct > 95,
      "normalized":   round(pct / 100, 3)
    })

  if isinstance(data.get("network"), dict):
    sent = data["network"].get("bytes_sent", 0)
    data["network"]["high_outbound"] = sent > 1_000_000  # 1 MiB threshold

  if isinstance(data.get("filesystem"), dict):
    changes = data["filesystem"].get("changes", [])
    data["filesystem"]["rapid_activity"] = len(changes) > 10

  return data
```

3.2 utils/scoring.py

Typical evolved version (rule-based, weighted, thresholded)

Python

```
# /opt/data/monitor/utils/scoring.py
```

```
from typing import Dict, Any, Tuple, List

WEIGHTS = {
    "cpu":        0.25,
    "network":    0.35,
    "filesystem": 0.25,
    "threads":    0.10,
    "events":     0.05
}

def calculate_risk_score(telemetry: Dict[str, Any]) -> Tuple[float, List[str], str]:
    score = 0.0
    reasons: List[str] = []

    # CPU rules
    cpu = telemetry.get("cpu", {})
    if cpu.get("is_high"):
        score += 0.30 * WEIGHTS["cpu"]
        reasons.append(f"High CPU usage ({cpu.get('system_cpu_percent', '?.?')}% > 85)")
    if cpu.get("is_critical"):
        score += 0.45 * WEIGHTS["cpu"]
        reasons.append("Critical CPU usage (>95%)")

    # Network rules
    net = telemetry.get("network", {})
    if net.get("high_outbound"):
        score += 0.50 * WEIGHTS["network"]
        reasons.append(f"Large outbound transfer ({net.get('bytes_sent', 0):,} bytes)")
```

```
    # Filesystem rules
    fs = telemetry.get("filesystem", {})
    if fs.get("rapid_activity"):
        score += 0.55 * WEIGHTS["filesystem"]
        reasons.append(f"Rapid filesystem activity ({len(fs.get('changes', []))} changes)")

    # TODO: add more rules from other telemetry keys as desired

    score = min(max(score, 0.0), 1.0)

    if score >= 0.80:
        label = "HIGH"
    elif score >= 0.45:
        label = "MEDIUM"
    else:
        label = "LOW"

    return score, reasons, label
```

3.3 utils/reporting.py

Readable, auditor-friendly version

Python

```
# /opt/data/monitor/utils/reporting.py

from typing import Dict, Any, List

def generate_human_explanation(
    telemetry: Dict[str, Any],
```

```
    score: float,
    reasons: List[str],
    label: str
) -> str:
    lines = [
        f"Behavioral Risk Classification: **{label}** (confidence score: {score:.1%})",
        "_____",
        "Observed indicators:",
    ]

    cpu = telemetry.get("cpu", {})
    if cpu.get("is_high"):
        lines.append(f"  • Elevated CPU utilization ({cpu.get('system_cpu_percent', '?.?')}%)")

    net = telemetry.get("network", {})
    if net.get("high_outbound"):
        lines.append(f"  • Significant outbound network volume detected")

    fs = telemetry.get("filesystem", {})
    if fs.get("rapid_activity"):
        lines.append(f"  • High filesystem modification rate ({len(fs.get('changes', []))} events)")

    if reasons:
        lines.extend(["", "Primary risk contributors:"])
        lines.extend([f"  • {r}" for r in reasons])

    lines.extend([
```

```
        "",
        "Recommended action:",
    ])

    if score >= 0.80:
        lines.append("→ Immediate suspension / forensic capture advised")
    elif score >= 0.45:
        lines.append("→ Manual review required within next 5–10 minutes")
    else:
        lines.append("→ Currently consistent with benign student/research code")

    return "\n".join(lines)
```

4. Typical Integration Snippet in monitor.py (main loop)

Python

```
# fragment from /opt/data/monitor/monitor.py main loop

while running:
    try:
        telemetry = collect_all_telemetry()          # ← enriched version
        score, reasons, label = calculate_risk_score(telemetry)
        explanation = generate_human_explanation(telemetry, score, reasons, label)

        # persist (already in skeleton – now with much richer document)
        db_collection.insert_one({
            "run_id": current_run_id,
            "timestamp": telemetry["timestamp_unix"],
            "iso": telemetry["iso_utc"],
            "raw_telemetry": telemetry,
```

```
        "risk": {
            "score": score,
            "label": label,
            "reasons": reasons
        },
        "explanation_text": explanation
    })

    print(f"[{telemetry['iso_utc']}] {label:<6} {score:5.1%}  {explanation.splitlines()[0]}")

  except Exception as e:
    logger.error(f"Monitoring cycle failed: {e}")

  time.sleep(poll_interval)   # e.g. 1.8 – 3.0 seconds
```

5. Final Summary Sentences

Lab 10 is a **runtime behavioural analysis mini-product**. The student's job is to turn a minimal skeleton into something that collects meaningful signals, fuses them into risk decisions, and explains those decisions in plain language — exactly what container security platforms have been evolving toward since the early 2020s.

The confusion was resolved by establishing that:

- **editing = always comp40771_container /opt/data/monitor/**
- **changes = deliberate extensions** (new telemetry getters + indicators + scoring rules + richer explanations)
- **value = quality & thoughtfulness** of those extensions, not correctness of Docker compose

This document contains every major code artefact that appeared during the resolution process.

Let me know which part you want to refine, test, extend or document further.